Stock Shark

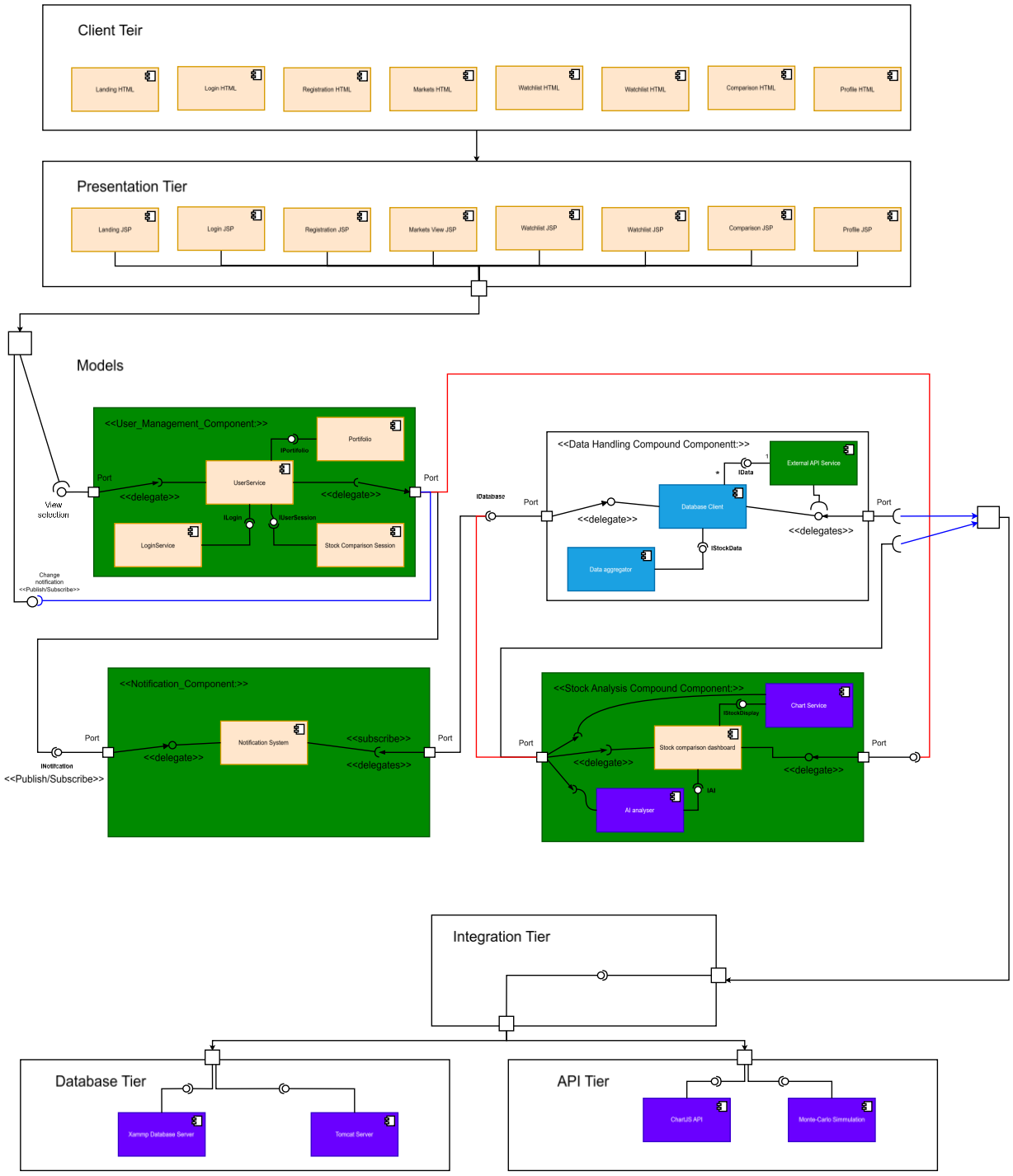
THE SOFTWARE ARCHITECTURE & DESIGN

5 Tier Architecture Component Diagram:



Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN



Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

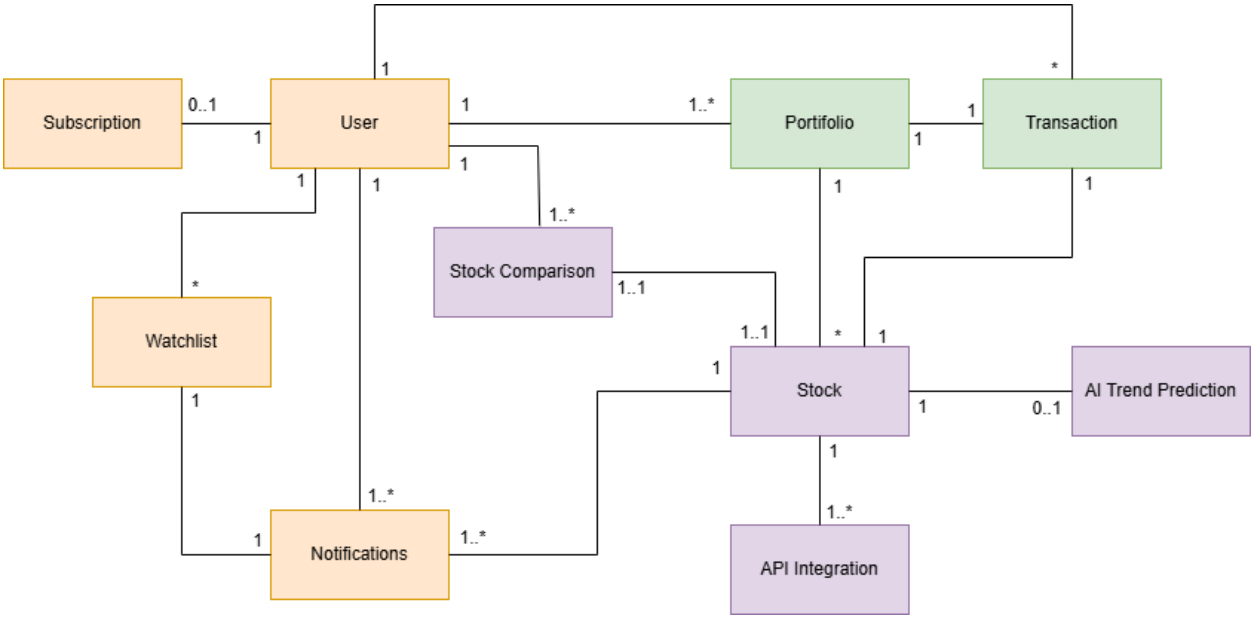
Building The Architecture

StockShark's architecture is built upon the core principles of Clean Architecture, which aims to create systems that are independent of frameworks, testable, and maintainable. This design follows the SOLID principles, making the system strong and easy to change.

- Single/Closed Principle (OCP): StockShark is designed to be open for extension but closed for modification. New features are added by introducing new code, rather than altering existing, stable code.
- Dependency Inversion Principle (DIP): High level modules, such as use cases that implement business logic, do not depend on low level modules, such as data access layers. Instead, low level modules depend on abstractions defined by high level modules.

By separating these layers, we create a system that's flexible, maintainable, and easy to understand. This makes it easier for our team to add new features, fix bugs, and adapt to changing requirements, ultimately providing you with a better StockShark experience.

Business Concept Model

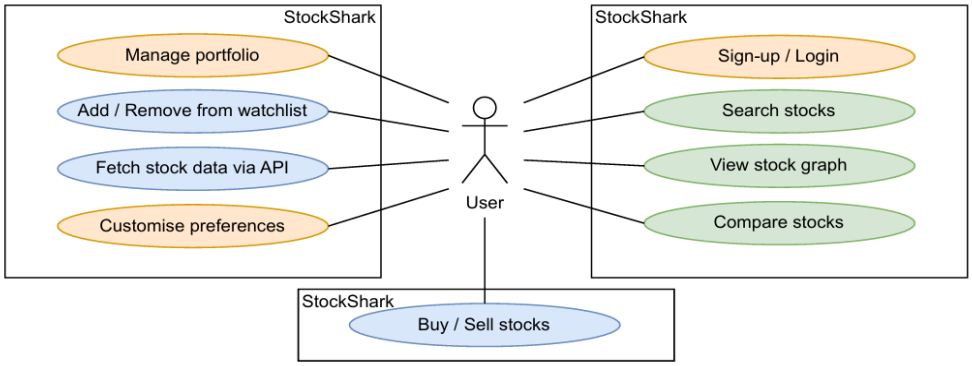


Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

The Business Concept Model is structured to provide a user-centric, scalable, and modular system for managing stock investments and transactions. At its core, the User entity connects to essential functions like portfolios, transactions, subscriptions, watchlists, and notifications, ensuring seamless interaction with the platform. The Stock entity is deeply integrated with real-time market updates via API integration, AI trend predictions, and stock comparisons, allowing for data-driven decision-making. The model also enforces flexibility, allowing users to manage multiple portfolios and transactions while ensuring that each action is properly tracked. The relational structure supports future expansion, such as adding new AI models, additional investment tools, or automation, without disrupting existing functionalities. By balancing functionality and scalability, this design enables a robust financial system that caters to both beginner and advanced investors.

Use Case Model



The StockShark Use Case Diagram is structured to provide a clear and user-centric experience, dividing functionalities into three main categories: portfolio management, stock exploration, and trading transactions. Users can manage their investments by adding or removing stocks from their watchlist, customizing preferences, and fetching real-time stock data via API. The system also supports stock research through features like stock search, comparison, and historical graph visualization. Additionally, the buy/sell stocks functionality ensures seamless trading operations. The inclusion of sign-up/login enhances security and access control. This structured approach ensures scalability, allowing for future enhancements like AI-driven insights or automated trading while maintaining an intuitive and efficient user workflow.

Stock Shark

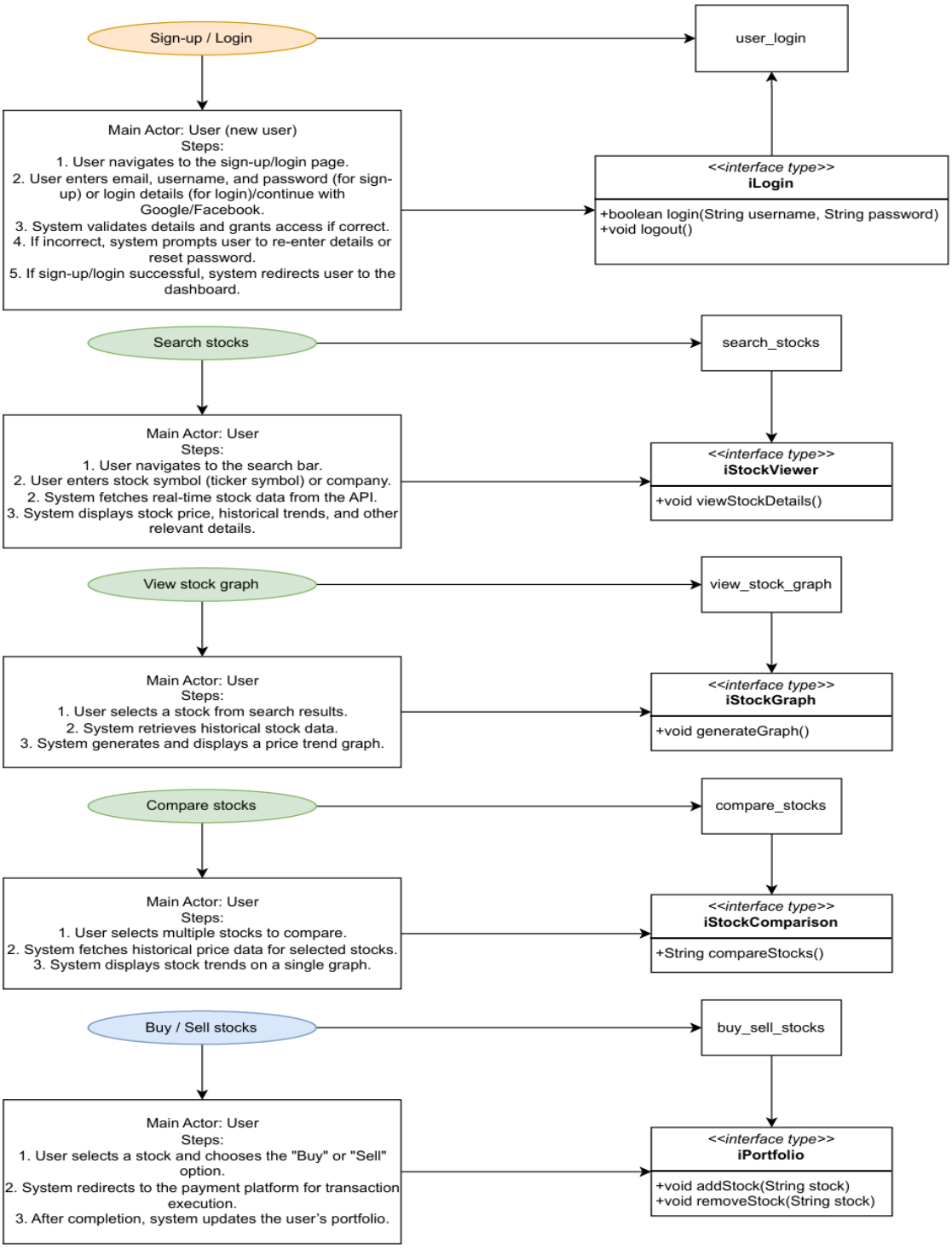
THE SOFTWARE ARCHITECTURE & DESIGN

System Interfaces



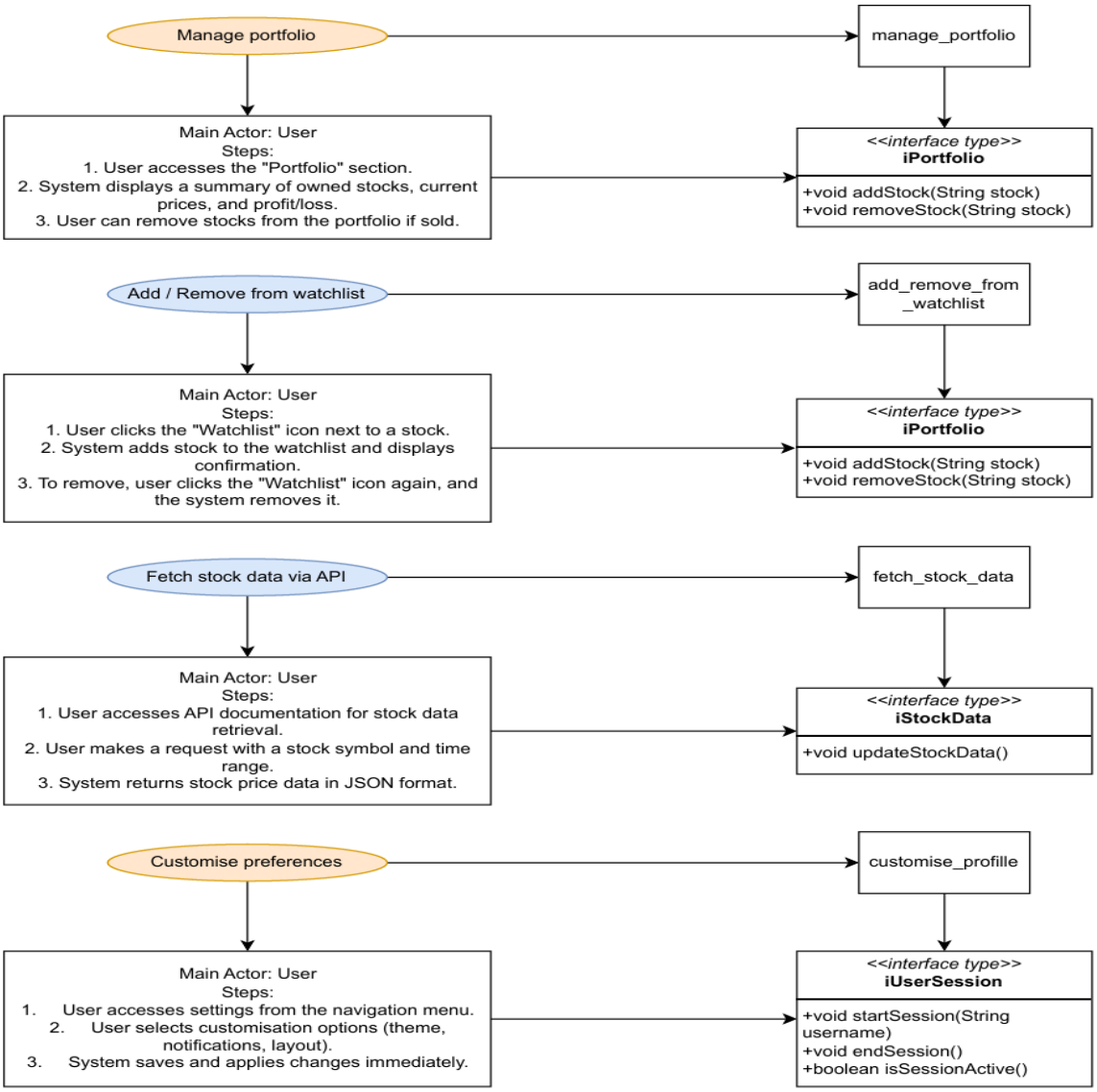
Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN



Stock Shark

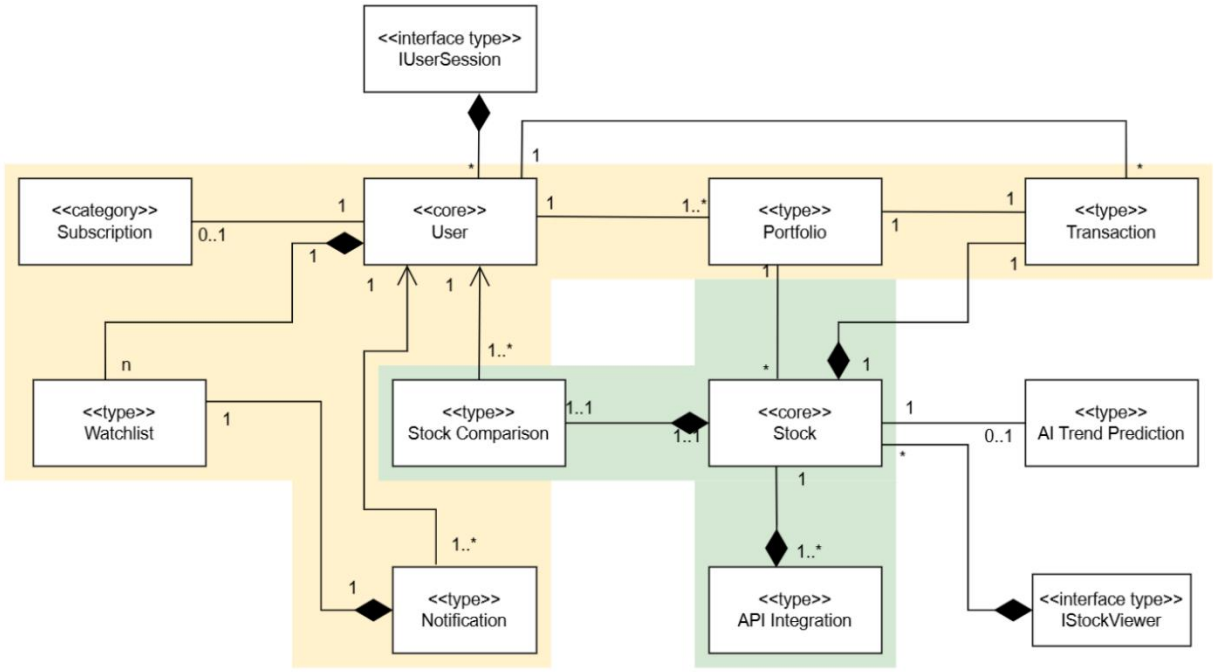
THE SOFTWARE ARCHITECTURE & DESIGN



Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

Business Type Model

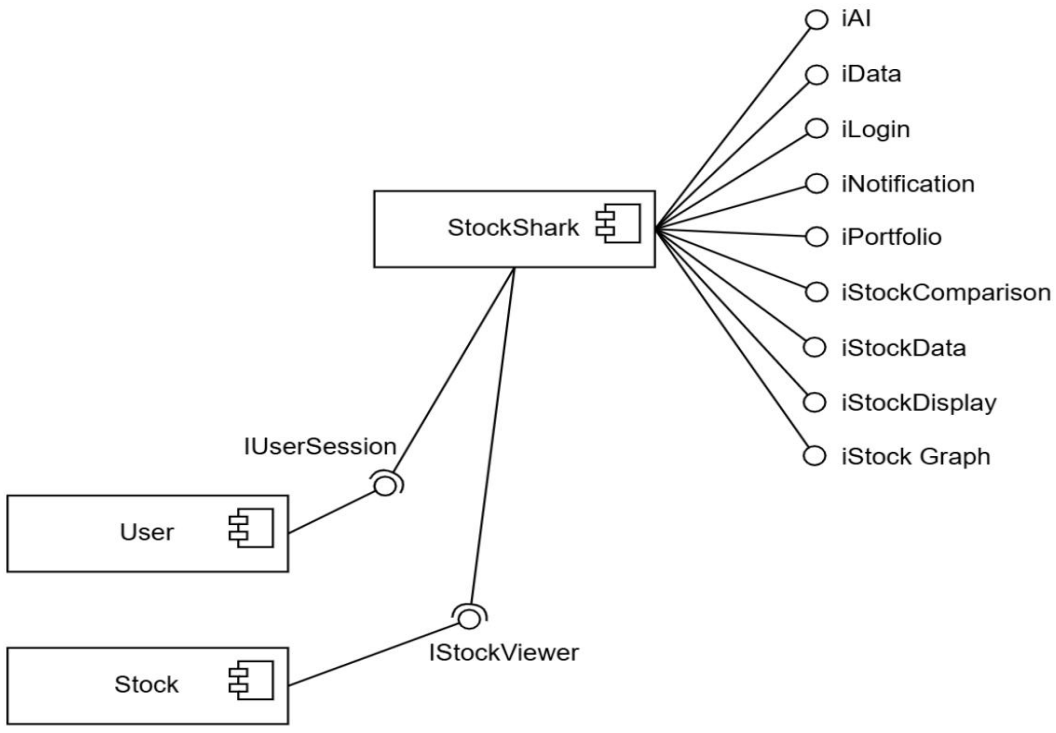


The Business Type Model in the diagram is structured to represent a scalable and modular stock trading system with distinct core entities, functional types, and interfaces. At the center, the User entity acts as the primary connection point, interacting with subscriptions, portfolios, transactions, notifications, and watchlists. The model ensures that users can manage their investments efficiently while maintaining real-time stock tracking through API integrations. The Stock entity, another core component, connects to stock comparison, AI trend prediction, and API integration, ensuring data-driven decision-making. Additionally, interface types like IUserSession and IStockViewer support session management and stock visualization, enhancing the platform's usability. The composition relationships (diamonds) indicate ownership structures, ensuring that key elements like portfolios, watchlists, and notifications are user-specific, while stocks exist independently. This model is designed for flexibility and expansion, allowing future enhancements such as automated trading, advanced analytics, or AI-driven insights without disrupting the core architecture.

Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

Initial System Architecture



The Initial System Architecture diagram represents the foundational structure of the StockShark platform, showcasing key system components and their interactions. At the core, StockShark serves as the central system, connecting various interfaces such as iAI, iData, iLogin, iNotification, iPortfolio, iStockComparison, iStockData, iStockDisplay, and iStockGraph, ensuring modularity and extendability. The User entity is linked through the IUserSession interface, allowing authentication and session management, ensuring secure access to the system. Meanwhile, the Stock entity interacts with StockShark via the IStockViewer interface, enabling real-time stock data retrieval and visualization. This architecture provides a clear separation of concerns, ensuring scalability, maintainability, and integration of advanced functionalities like AI-based trend analysis and stock comparison. The structured design allows for future enhancements, such as automated trading or additional analytics, while keeping the system flexible and user-friendly.